WanderMate: Social Mobile Travel Planner

Project 3 — Task 3 Deliverable

Architectural Tactics & Design Patterns

This document outlines five architectural tactics and two design patterns employed in the WanderMate system, explaining how each addresses specific non-functional requirements and contributes to the overall quality attributes of the architecture.

Contents

---

1 Introduction **2**

2 Architectural Tactics **2**

2.1 Tactic 1: Multi-layer Response Caching 2

2.1.1 Description 2

2.1.2 Non-Functional Requirements Addressed 2

2.1.3 Implementation Details 2

2.1.4 Caching Flow Diagram 4

2.1.5 Reasoning 5

2.2 Tactic 2: Authentication & Secret Proxying 5

2.2.1 Description 5

2.2.2 Non-Functional Requirements Addressed 5

2.2.3 Implementation Details 5

2.2.4 Security Architecture Diagram 6

2.2.5 Reasoning 6

2.3 Tactic 3: Rate Limiting & Throttling 6

2.3.1 Description 6

2.3.2 Non-Functional Requirements Addressed 7

2.3.3 Implementation Details 7

2.3.4 Reasoning 7

2.4 Tactic 4: Graceful Degradation & Fallback 7

2.4.1 Description 7

2.4.2 Non-Functional Requirements Addressed 8

2.4.3 Implementation Details 8

2.4.4 Degradation Flow Diagram 9

2.4.5 Reasoning 9

2.5 Tactic 5: Separation of Concerns via Layered Architecture 9
2.5.1 Description 9
2.5.2 Non-Functional Requirements Addressed 10
2.5.3 Implementation Details 10
2.5.4 Layered Architecture Diagram 10
2.5.5 Reasoning 10
2.6 Architectural Tactics — Summary 11

3 Implementation Patterns **11**
3.1 Pattern 1: Facade + Strategy (TravelService) 11
3.1.1 Pattern Description 11
3.1.2 Problem Context 11
3.1.3 Solution Applied 12
3.1.4 UML Class Diagram 12
3.1.5 Role Within the Architecture 12
3.2 Pattern 2: Observer Pattern (Real-time Collaboration) 13
3.2.1 Pattern Description 13
3.2.2 Problem Context 13
3.2.3 Solution Applied 13
3.2.4 UML Class Diagram 14
3.2.5 Sequence of a Collaborative Edit 15
3.2.6 Role Within the Architecture 15

4 Relationship Between Tactics and Patterns **16**

5 Conclusion **18**

1. Introduction

WanderMate is a social, collaborative mobile travel planner built as a React Native (Expo) frontend communicating with a Node.js/Express backend, backed by MongoDB for persistent storage and Firebase Realtime Database for real-time event propagation. The system integrates multiple external APIs — Google Places (POI discovery), OpenRouteService (routing and optimization), and Overpass/OSM (disabled fallback) — all proxied through a backend API Gateway.

The architecture must simultaneously satisfy demanding non-functional requirements spanning **performance**, **security**, **availability**, **scalability**, and **modifiability**, while operating within the economic constraints of free-tier API quotas. This document catalogues the **five architectural tactics** and **two design patterns** that form the backbone of WanderMate's architectural strategy, linking each to specific NFRs and illustrating their implementation with diagrams.

> **Notation.** Throughout this document, requirement identifiers such as NFR-06 and FR-07 refer to the requirements defined in the Task 1 deliverable. Code references use `monospace` font and point to files in the WanderMate repository.

2. Architectural Tactics

Architectural tactics are fundamental design decisions that directly influence a system's quality attributes. Each tactic below addresses one or more non-functional requirements and is implemented in specific components of the WanderMate codebase.

2.1 Tactic 1: Multi-layer Response Caching

2.1.1 Description

Response caching is a **performance tactic** that reduces latency and resource consumption by storing the results of expensive computations or external API calls and serving them directly on subsequent identical requests, bypassing the original computation entirely. WanderMate implements caching at *three distinct architectural layers*, creating a defence-in-depth strategy against latency spikes and API quota exhaustion.

2.1.2 Non-Functional Requirements Addressed

- **NFR-01 (Performance)** — Ensures POI search results and map data are returned within the 2-second target at p95 by serving cached responses for repeated queries.
- **NFR-06 (Scalability / Quota Compliance)** — Minimises outbound calls to Google Places ($\leq$ 5,000 req/month) and ORS ($\leq$ 2,000 req/day) by caching responses with calibrated TTLs, ensuring zero overage cost at launch.
- **NFR-07 (Availability)** — Client-side caching via AsyncStorage allows itinerary data to be read even when the device is completely offline.

2.1.3 Implementation Details

Layer	Technology	Scope & TTL	Code Location
L1: Client	AsyncStorage via `localCache` module	Trip data (infinite TTL offline), general cache (1 hour default TTL)	`frontend/services/syncQueue.ts`
L2: Service	In-process `node-cache` inside TravelService	POI search: 30 min, Place details: 24 hr, Routes: 1 hr	`backend/src/services/travelService`
L3: Middleware	Express middleware `cacheMiddleware`	Per-route response cache with configurable TTL (default 1 hr)	`backend/src/middleware/cache.js`

2.1.4 Caching Flow Diagram

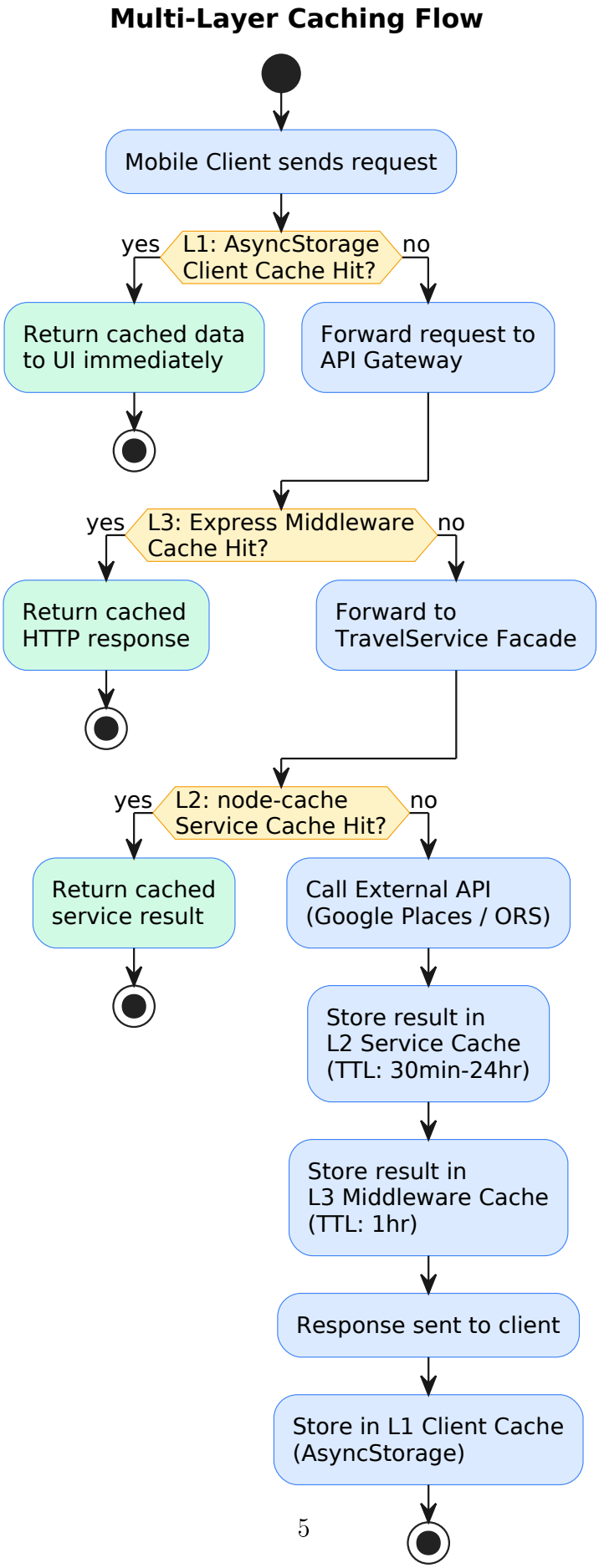

Figure 1: Multi-layer caching flow: requests traverse L1 (client), L2 (service), and L3

2.1.5 Reasoning

The three-layer caching strategy is calibrated to the specific cost and latency characteristics of each external API. Google Places, with its restrictive free-tier quota (5,000 requests/month), receives the longest cache TTL for place details (24 hours), because details data (address, opening hours, photos) changes infrequently. Route calculations via ORS receive a moderate TTL (1 hour), balancing freshness against the daily quota cap. Client-side caching with infinite TTL during offline mode satisfies NFR-07 by ensuring previously viewed trips remain accessible without network connectivity.

2.2 Tactic 2: Authentication & Secret Proxying

2.2.1 Description

Authentication and secret proxying is a **security tactic** that prevents unauthorised access and protects sensitive credentials from exposure. WanderMate employs a dual-layer security model: *Firebase JWT token verification* authenticates every API request, and the *API Gateway pattern* ensures that all third-party API keys reside exclusively on the server, never reaching the mobile client bundle.

2.2.2 Non-Functional Requirements Addressed

- **NFR-10 (Security / Transport)** — All client–server communication uses HTTPS with TLS 1.2+. The API Gateway additionally applies Helmet security headers to every response.
- **NFR-11 (Security / Secret Protection)** — Third-party API keys (Google Places, ORS) are stored as server-side environment variables in `.env` and are never transmitted to the client. All calls to external services are proxied through the backend gateway.
- **NFR-12 (Security / Authorisation)** — Per-trip access control (ownership + collaborator checks) is enforced at the API layer, preventing horizontal privilege escalation.

2.2.3 Implementation Details

1. **Token-based Authentication**: The `authenticate` middleware (`backend/src/middleware/auth.js`) extracts the Bearer token from the `Authorization` header of every request and verifies it using Firebase Admin SDK's `verifyIdToken()` method. The decoded claims (UID, email, display name) are attached to `req.user` for downstream route handlers.
2. **Client-side Token Injection**: The Axios request interceptor in `frontend/services/api.ts` automatically attaches the current user's Firebase ID token to every outbound API call, ensuring seamless authentication without manual token management in UI components.
3. **Secret Vault**: All third-party API keys (`GOOGLE_PLACES_API_KEY`, `ORS_API_KEY`) are loaded from environment variables in the backend process. The mobile client communicates only with the WanderMate API Gateway; it has no knowledge of — and no access to — external API credentials.
4. **HTTP Security Headers**: The `helmet` middleware is applied globally to set `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, and other protective headers on every response.

2.2.4 Security Architecture Diagram

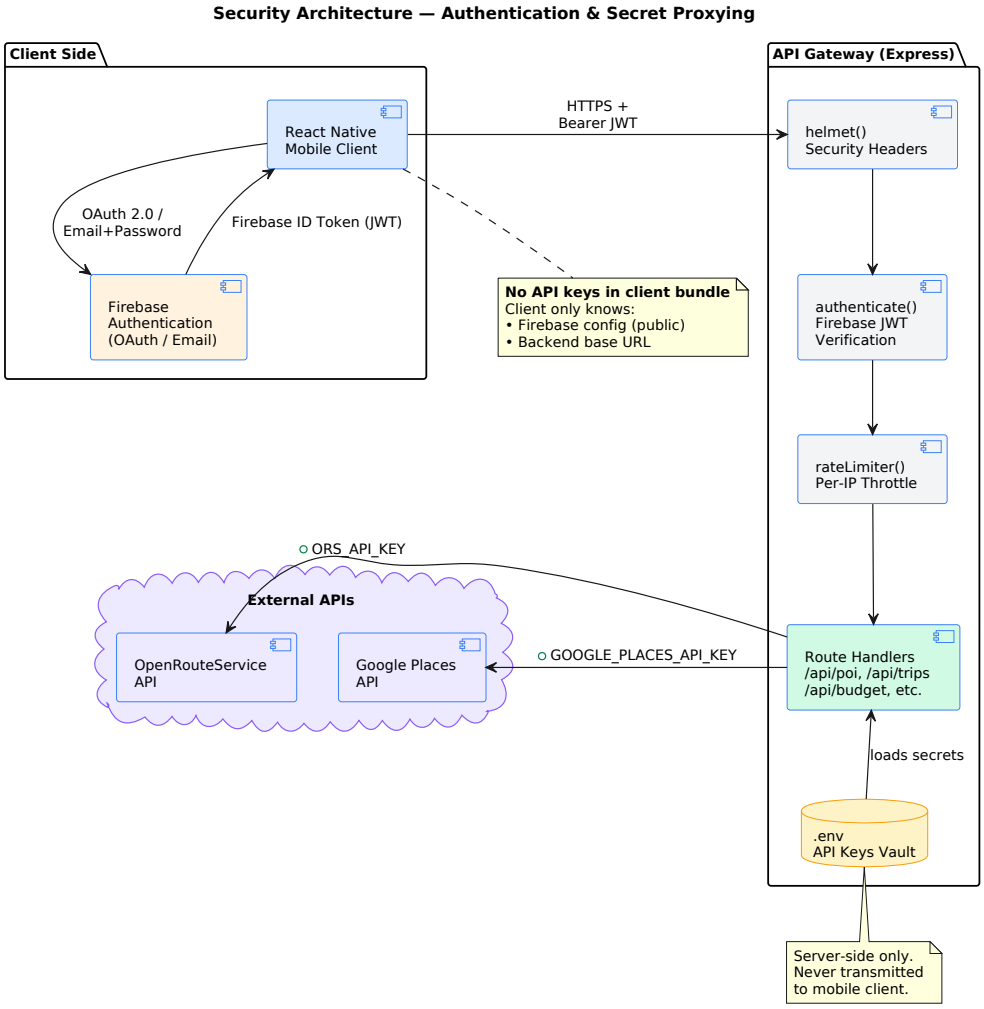

Figure 2: Security architecture: the mobile client authenticates via Firebase JWT tokens; all external API calls are proxied through the backend gateway which injects API keys from server-side environment variables. Direct client-to-external-API communication is blocked by design.

2.2.5 Reasoning

Embedding API keys in a mobile application bundle is a well-documented anti-pattern: reverse engineering tools (e.g., `apktool`) can extract keys from Android APKs in minutes. By routing all external API calls through the backend gateway, WanderMate ensures that keys are stored only in server memory. The gateway simultaneously serves as a centralised enforcement point for authentication, rate limiting, and caching — three concerns that would be fragmented and unreliable if handled on the client side.

2.3 Tactic 3: Rate Limiting & Throttling

2.3.1 Description

Rate limiting is a **scalability tactic** that controls the volume of requests processed per unit of time, protecting both the system's own resources and the quotas of external

services. WanderMate applies rate limiting at two granularities: a *general limiter* for all API endpoints and a stricter *external API limiter* for routes that proxy requests to third-party services.

2.3.2 Non-Functional Requirements Addressed

- **NFR-04 (Scalability)** — Prevents any single user from monopolising server resources, ensuring the gateway can sustain 200 concurrent users.
- **NFR-06 (Quota Compliance)** — The external API rate limiter directly caps the number of outbound calls to Google Places and ORS, guaranteeing zero overage cost at launch.

2.3.3 Implementation Details

Limiter	Window	Max Requests	Scope / Applied To
`generalLimiter`	15 minutes	100 per IP	All `/api/*` endpoints (global middleware in `index.js`)
`externalApiLimit`	1 hour	50 per IP	POI search, category browsing, place details, route calculation (applied per-route in `poi.js` and `routes.js`)

Both limiters use the `express-rate-limit` library with `standardHeaders: true` (returning `RateLimit-*` response headers) and `legacyHeaders: false`. When a client exceeds the limit, the middleware returns a JSON error response:

```
HTTP 429 Too Many Requests
{ "error": "Too many requests, please try again later" }
```

2.3.4 Reasoning

The dual-tier rate limiting strategy reflects the asymmetric cost structure of WanderMate's backend operations. Internal operations (trip CRUD, budget management) are cheap — they hit only MongoDB. External operations (POI search, route optimisation) incur real costs against finite free-tier quotas. By applying a much tighter limit (50 req/hour) to external API proxying routes, the system ensures that even under aggressive usage patterns, daily/monthly quotas are not exhausted. The general limiter (100 req/15 min) serves as a broad denial-of-service mitigation layer, protecting the Express event loop from resource exhaustion.

2.4 Tactic 4: Graceful Degradation & Fallback

2.4.1 Description

Graceful degradation is an **availability tactic** that ensures the system continues to provide useful functionality when individual components or external dependencies fail. Rather than surfacing errors or blank screens to users, the system falls back to alternative data sources or cached data, providing a degraded but functional experience.

2.4.2 Non-Functional Requirements Addressed

- **NFR-07 (Availability / Offline-first)** — Core itinerary viewing and editing remain available when the device is offline, using AsyncStorage-backed local cache and a sync queue for pending mutations.
- **NFR-08 (Reliability / API Failure)** — When Google Places API is unavailable, POI search degrades gracefully to ORS POI results with zero user-facing crashes.
- **NFR-09 (Reliability / Error Handling)** — Every failure scenario returns a structured JSON error response; uncaught exceptions are captured by the global Express error handler.

2.4.3 Implementation Details

WanderMate implements graceful degradation at three levels:

1. **Server-side Provider Fallback**: The `TravelService.searchPOI()` method wraps the Google Places API call in a `try/catch`. On failure (network timeout, quota exhaustion, invalid response), it silently falls through to the ORS POI endpoint:
```
results = await this.places.searchPOI(...)
                 .catch(err => { return []; });
if (results.length === 0) {
    results = await this.ors.searchPOI(...)
                   .catch(err => { return []; });
}
```

2. **Client-side Offline Fallback**: Every Zustand store method (e.g., `fetchTrips`, `fetchTrip`) first attempts an API call. On failure, it reads from `localCache` (AsyncStorage) with infinite TTL, displaying the last-known state:
```
try { response = await api.get('/trips'); }
catch {
    const cached = await localCache.get('trips', Infinity);
    if (cached) set({ trips: cached });
}
```

3. **Mutation Queuing**: When write operations fail (offline or server error), the `syncQueue` persists the pending mutation to AsyncStorage. On reconnect, the queue flushes in timestamp order, ensuring eventual consistency:
```
catch (error) {
    await syncQueue.add({
      method: 'POST',
      url: '/trips',
      data
    });
}
```

2.4.4 Degradation Flow Diagram

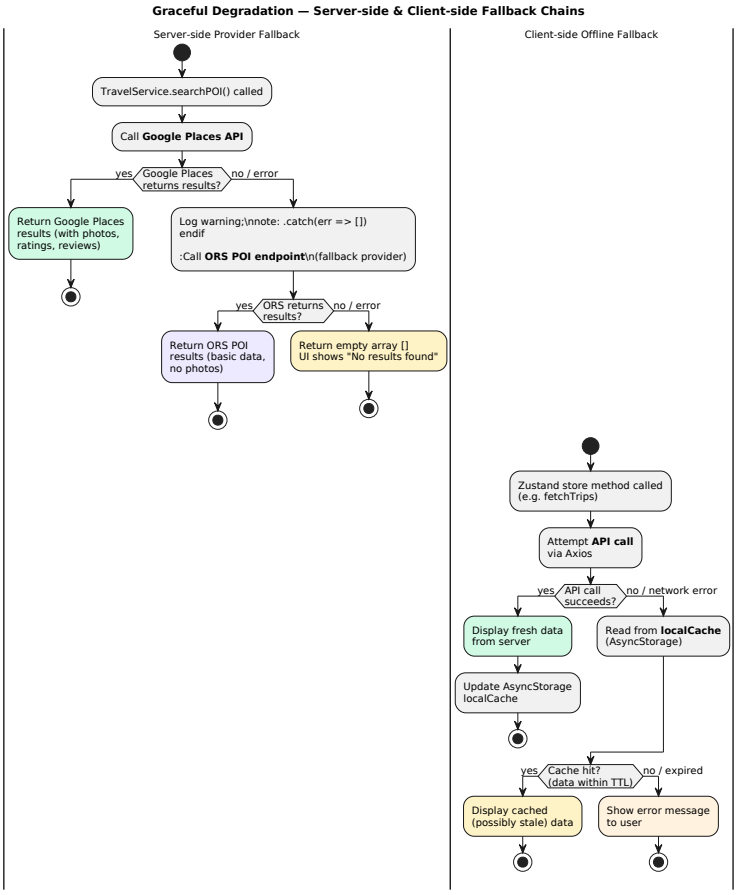

Figure 3: Graceful degradation chains. Left: server-side POI provider fallback (Google Places → ORS → empty). Right: client-side offline fallback (API → AsyncStorage cache → error message).

2.4.5 Reasoning

Travel applications must remain usable in environments with intermittent connectivity (airports, remote destinations, underground transport). By treating offline access as a *first-class architectural concern* rather than an edge case, WanderMate ensures that users can always view and edit their itineraries. The server-side provider fallback chain prevents a single external API outage from rendering the entire POI discovery feature unusable — a critical resilience property for a system that depends on third-party services.

2.5 Tactic 5: Separation of Concerns via Layered Architecture

2.5.1 Description

Separation of concerns is a **modifiability tactic** that partitions the system into layers with well-defined responsibilities and strict dependency direction. Each layer communicates only with its immediate neighbours, preventing cross-layer coupling and enabling independent evolution of UI, business logic, and data access code.

2.5.2 Non-Functional Requirements Addressed

- **NFR-14 (Maintainability / Modularity)** — The codebase is organised into independently testable modules aligned with subsystem boundaries, with module fan-out constrained to fewer than 5 dependencies per module.
- **NFR-15 (Maintainability / Provider Swappability)** — Swapping the POI provider requires changes only within the TravelService Facade and the specific adapter module; no changes to UI screens, routes, or stores are necessary.

2.5.3 Implementation Details

WanderMate employs a four-layer architecture on both the backend and frontend:

Backend layers (Node.js / Express):

1. **Presentation Layer** — Express Route Handlers (`trips.js`, `poi.js`, `budget.js`, `feed.js`, `users.js`, `routes.js`)
2. **Business Logic Layer** — TravelService Facade, TripObserver (EventEmitter)
3. **Data Access Layer** — Mongoose Models (`Trip`, `Expense`, `FeedPost`, `User`)
4. **Infrastructure Layer** — Config, Middleware (`auth`, `cache`, `rateLimiter`), Firebase Admin SDK

Frontend layers (React Native / Expo):

1. **View Layer** — React Native Screens and Components (`app/`, `components/`)
2. **State Management Layer** — Zustand Stores (`tripStore`, `budgetStore`, `feedStore`, `authStore`)
3. **Service Layer** — API Client (`api.ts`), Sync Queue (`syncQueue.ts`), Local Cache
4. **Infrastructure Layer** — Firebase SDK (`config/firebase.ts`), AsyncStorage, Env Config

2.5.4 Layered Architecture Diagram

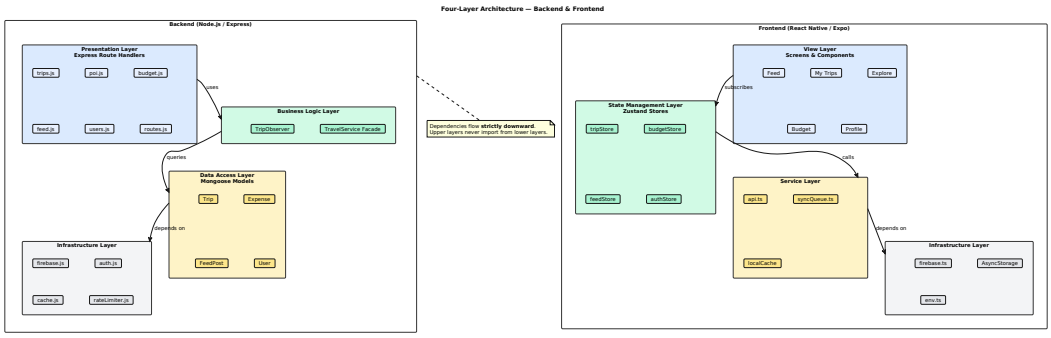

Figure 4: Four-layer architecture on both backend and frontend. Dependencies flow strictly downward; upper layers depend on lower layers but never vice versa.

2.5.5 Reasoning

The strict layered dependency direction enforces a critical maintainability invariant: changes in infrastructure (e.g., replacing AsyncStorage with MMKV, migrating from node-cache to Redis) propagate upward through at most one layer boundary. Similarly, UI redesigns affect only the view layer. This was validated when the Overpass adapter was disabled: the entire change was confined to the TravelService constructor (busi-

ness logic layer) — no route handler, store, or UI component required modification, satisfying NFR-15.

2.6 Architectural Tactics — Summary

#	Tactic	Quality Attr.	NFRs Addressed	Key Implementation
1	Multi-layer Response Caching	Performance, Scalability	NFR-01, NFR-06, NFR-07	`localCache`, `node-cache`, `cacheMiddleware`
2	Authentication & Secret Proxying	Security	NFR-10, NFR-11, NFR-12	Firebase JWT, `authenticate` middleware, `.env` vault
3	Rate Limiting & Throttling	Scalability	NFR-04, NFR-06	`generalLimiter`, `externalApiLimiter`
4	Graceful Degradation & Fallback	Availability, Reliability	NFR-07, NFR-08, NFR-09	Provider fallback chain, `syncQueue`, offline cache
5	Separation of Concerns (Layered Architecture)	Modifiability	NFR-14, NFR-15	4-layer backend/frontend, strict dependency direction

3. Implementation Patterns

This section describes two design patterns central to WanderMate's architecture, including their roles, class relationships, and implementation details. Each pattern is illustrated with a UML class diagram.

3.1 Pattern 1: Facade + Strategy (TravelService)

3.1.1 Pattern Description

The **Facade** pattern provides a unified, simplified interface to a complex subsystem, hiding internal complexity from clients. The **Strategy** pattern defines a family of algorithms (or service providers), encapsulates each one, and makes them interchangeable. WanderMate combines both patterns in the `TravelService` class, which serves as the single point of entry for all POI discovery and route optimisation operations.

3.1.2 Problem Context

WanderMate integrates three distinct external APIs for POI data (Google Places, ORS POIs, Overpass/OSM) and one for routing (ORS Directions). Each API has:

- A different HTTP protocol (REST, Overpass QL)
- A different response schema (JSON with varying field names and nesting)
- Different quota limits and pricing models

– Different failure modes (rate limiting, geographic restrictions, timeouts)

Without a unifying layer, each API consumer (Express route handler) would need to know which provider to call, how to normalise its response, when to fall back, and how to manage caching — dramatically increasing coupling and making provider changes ripple through the entire codebase.

3.1.3 Solution Applied

The `TravelService` Facade exposes four high-level methods (`searchPOI`, `searchByCategory`, `getPlaceDetails`, `getRoute`) that abstract away all provider-specific logic. Internally, it delegates to interchangeable adapter classes (`PlacesAdapter`, `ORSAdapter`, and the disabled `OverpassAdapter`), each of which implements the same normalised interface but targets a different external API. The Facade also encapsulates caching, fallback orchestration, and error handling.

3.1.4 UML Class Diagram

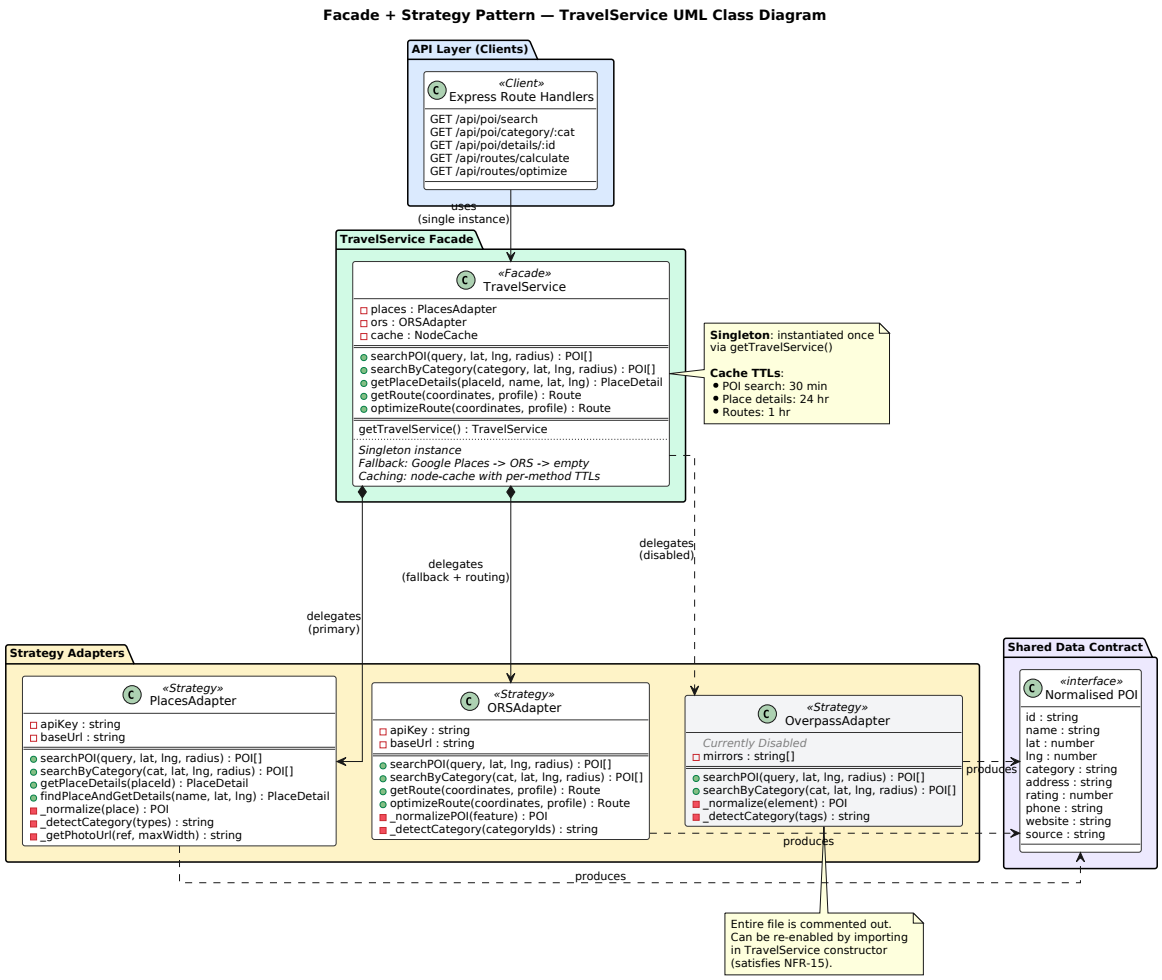

Figure 5: UML class diagram: Facade + Strategy pattern in TravelService. Route handlers interact only with the Facade; adapters are interchangeable and produce normalised POI objects.

3.1.5 Role Within the Architecture

Concern	How the Pattern Addresses It
Provider Independence	Route handlers call `travelService.searchPOI()` without knowing whether the response came from Google Places, ORS, or Overpass. Adding a new provider requires implementing a new adapter class and registering it in the Facade constructor.
Fallback Orchestration	The Facade's `searchPOI()` method implements the fallback chain (Google → ORS → empty). Adapters are unaware of each other; fallback logic is centralised in the Facade.
Cache Management	Caching is handled once in the Facade (using `node-cache` with per-operation TTLs), preventing duplicate caching logic across adapters and route handlers.
NFR-15 Compliance	When the Overpass adapter was disabled, zero changes were needed in route handlers or UI components — validating the pattern's information-hiding guarantee.

3.2 Pattern 2: Observer Pattern (Real-time Collaboration)

3.2.1 Pattern Description

The **Observer** pattern defines a one-to-many dependency between objects, so that when one object (the "subject") changes state, all dependent objects ("observers") are automatically notified and updated. WanderMate uses this pattern extensively for real-time collaborative features, leveraging Firebase Realtime Database as the event bus that connects publishers (backend mutation handlers) and subscribers (frontend Zustand stores).

3.2.2 Problem Context

WanderMate supports real-time collaborative trip editing: multiple users can simultaneously modify the same itinerary, and all participants must see changes propagated within 500 ms (NFR-03). This requires:

- A push-based notification mechanism (polling would violate NFR-03's latency target)
- Decoupled publishers and subscribers (the backend should not track connected clients)
- Multiple independent channels (trip updates, budget updates, auth state changes)
- Automatic cleanup on disconnect

3.2.3 Solution Applied

Firebase Realtime Database acts as the *subject* in the Observer pattern. On the backend, every trip or budget mutation writes an event to a Firebase path (e.g., `trips/{tripId}`). On the frontend, Zustand stores register `onValue` listeners on these paths — these are the *observers*. When Firebase detects a change, it pushes the update to all connected clients, which then refresh their local state.

3.2.4 UML Class Diagram

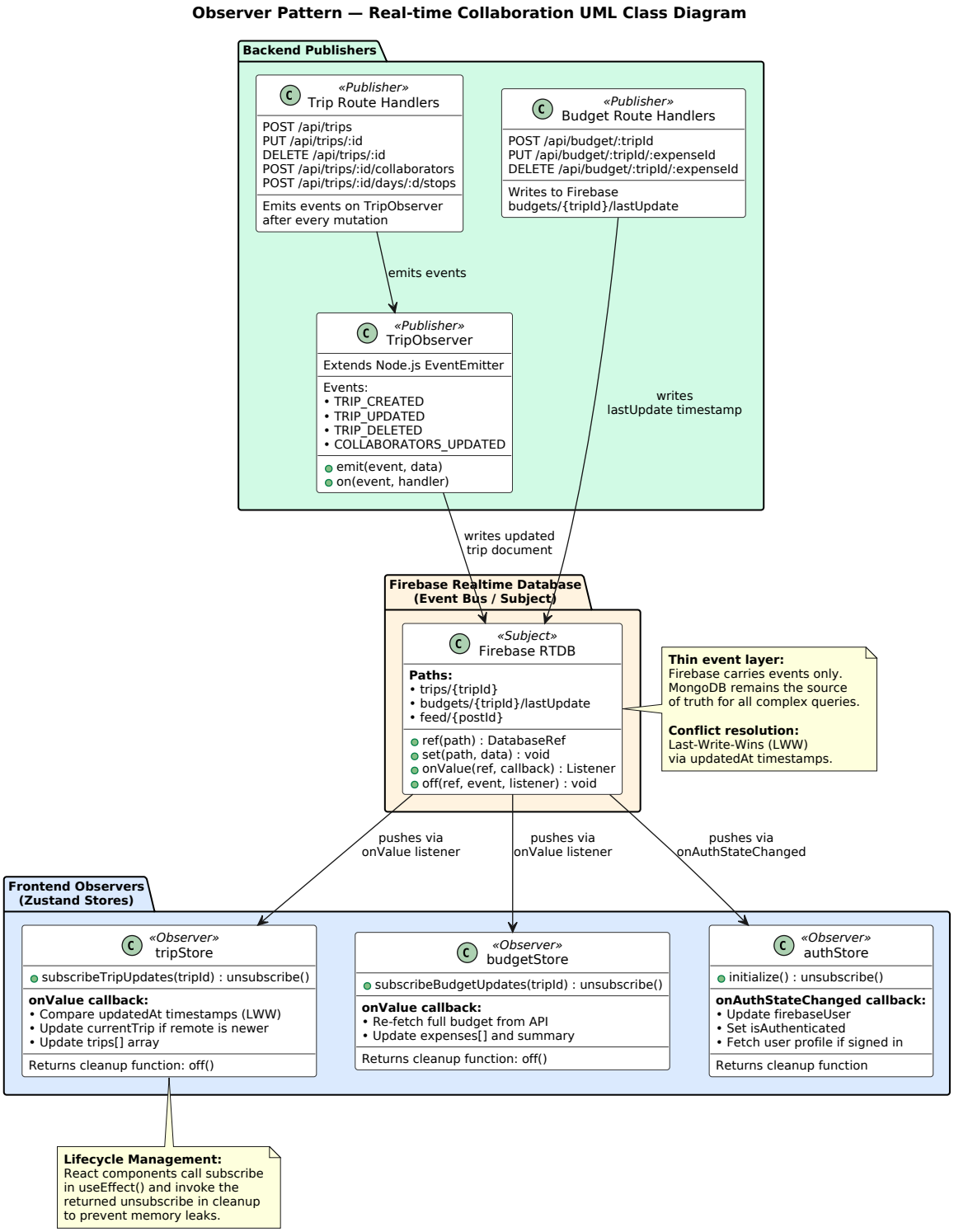

Figure 6: UML class diagram: Observer pattern for real-time collaboration. Backend publishers write events to Firebase paths; frontend stores subscribe via `onValue` listeners and receive push notifications. Each subscription returns a cleanup function (`off()`) to prevent memory leaks.

3.2.5 Sequence of a Collaborative Edit

The following sequence describes how a collaborative trip edit propagates to all connected clients:

1. **User A** edits a trip stop via the mobile client. The edit request is sent to the API Gateway.
2. The **API Gateway** processes the request: updates the Trip document in MongoDB and emits a `TRIP_UPDATED` event on the `TripObserver` EventEmitter.
3. The `TripObserver` event handler catches the event and writes the full updated trip document to `trips/{tripId}` in **Firebase Realtime Database**.
4. Firebase pushes the update to all connected clients that have registered `onValue` listeners on that path.
5. **User B's** `tripStore.subscribeTripUpdates()` callback fires with the new data. The store compares timestamps (Last-Write-Wins) and updates local state if the remote version is newer.
6. The React Native UI re-renders automatically via Zustand's reactive subscription model.

3.2.6 Role Within the Architecture

Concern	How the Pattern Addresses It
Real-time Sync (NFR-03)	Firebase's push-based event model ensures updates propagate within 500 ms, eliminating the latency overhead of polling.
Decoupled Communication	Publishers (backend route handlers) and subscribers (frontend stores) have no direct dependency on each other. The backend writes to Firebase paths without knowing which or how many clients are listening.
Multi-channel Events	Three independent observation channels (trip updates, budget updates, auth state) operate concurrently without interference, each managed by its own Firebase path and Zustand store.
Resource Cleanup	Every `subscribe*` method returns an `unsubscribe` function (wrapping Firebase's `off()` method). React components call this in their cleanup effect (`useEffect` return), preventing memory leaks and orphan listeners.
Conflict Resolution	The `tripStore` observer implements last-write-wins (LWW) by comparing `updatedAt` timestamps before applying incoming state: `if (remote.updatedAt >= local.updatedAt)` — providing eventual consistency without complex merge logic.

4. Relationship Between Tactics and Patterns

The architectural tactics and design patterns are not independent — they reinforce each other in a cohesive architectural strategy. The following diagram and table illustrate the relationships:

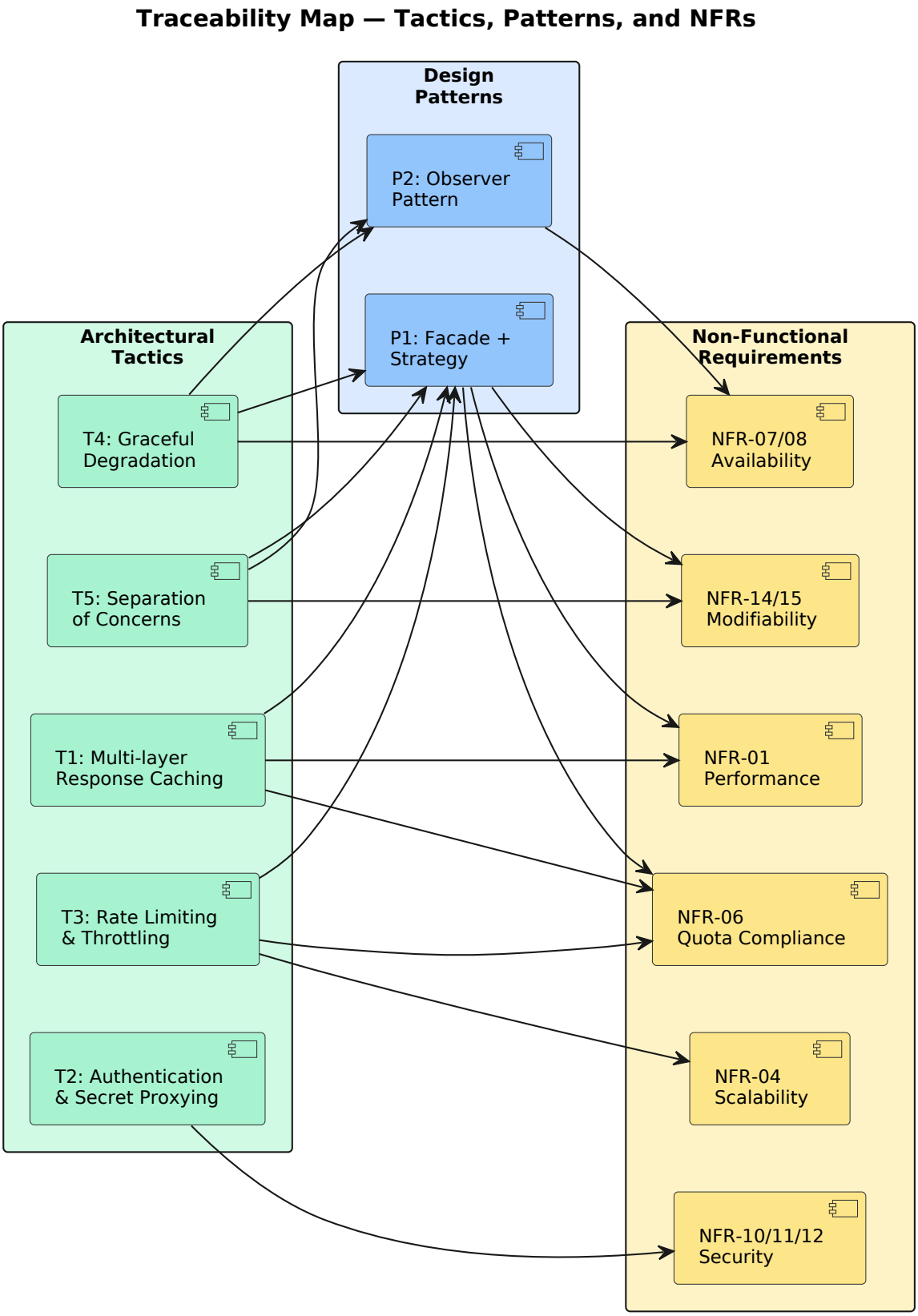

Figure 7: Traceability map: architectural tactics (green) and design patterns (blue) are linked to the non-functional requirements (amber) they jointly satisfy. Dashed lines show mutual reinforcement between tactics and patterns.

Tactic	Enabled By Pattern	Relationship
T1: Multi-layer Caching	Facade + Strategy	Caching is centralised in the Facade, applied once regardless of which adapter produced the result.
T3: Rate Limiting	Facade + Strategy	Rate limiting is applied at the gateway layer, before the Facade is invoked. The Facade's unified interface means a single rate limiter covers all provider calls.
T4: Graceful Degradation	Facade + Strategy	The fallback chain (Google → ORS → empty) is orchestrated inside the Facade. Adapters are unaware of fallback logic.
T4: Graceful Degradation	Observer Pattern	The Observer pattern enables real-time state refresh after reconnection, complementing the sync queue's offline mutation queuing.
T5: Layered Architecture	Both Patterns	Both patterns enforce strict layer boundaries: the Facade isolates adapters from route handlers; the Observer decouples publishers from subscribers via Firebase.

5. Conclusion

WanderMate's architecture is shaped by a deliberate set of five architectural tactics and two design patterns that collectively address the system's most demanding non-functional requirements:

1. **Multi-layer Response Caching** ensures sub-2-second response times and free-tier quota compliance through a three-tier cache hierarchy (client → service → middleware).
2. **Authentication & Secret Proxying** protects API keys from mobile bundle extraction and enforces Firebase JWT verification on every request via the API Gateway.
3. **Rate Limiting & Throttling** prevents resource exhaustion and quota overruns through dual-tier per-IP rate limits.
4. **Graceful Degradation & Fallback** ensures continuous availability through provider fallback chains and offline-first client caching with a persistent sync queue.
5. **Separation of Concerns via Layered Architecture** enforces strict dependency direction across four layers on both backend and frontend, enabling independent evolution and provider swappability.

The **Facade + Strategy** pattern (TravelService) unifies multiple external API providers behind a single interface, centralising caching, fallback, and quota management. The **Observer** pattern (Firebase Realtime Database subscriptions) enables real-time collaborative editing with sub-500ms propagation latency, decoupled publisher–subscriber communication, and automatic resource cleanup.

These tactics and patterns are not applied in isolation — they form a mutually reinforcing architectural strategy where the Facade centralises the application of caching, rate limiting, and degradation tactics, and the Observer pattern complements offline-first availability with real-time synchronisation on reconnect.